Procedimientos de desarrollo

Sistema de Gestión de Vuelos — AICM

Documento	Procedimientos, historial y decisiones de desarrollo
Versión	1.1
Fecha	2026-11-01
Sistema	v1.6.1
Repositorio	https://github.com/Demybrictany/Sistema-gestion-de-vuelos
Equipo	Juanjo, Demy

> Este documento explica **qué se construyó y cómo**: descripción, requerimientos, modelo de datos, metodología, versiones, decisiones, pruebas e infraestructura desplegada. Para usar el sistema, ver `01-MANUAL_DE_USUARIO.md`; para administrarlo, ver `02-MANUAL_ADMINISTRADOR_PROGRAMADOR.md`.

> **Novedades de la versión 1.1 del documento:** se agregaron la descripción y el alcance del proyecto, los requerimientos de negocio, usuario, funcionales, no funcionales y reglas de negocio, el modelo de datos con diagrama entidad-relación y la infraestructura desplegada (URLs, credenciales, variables, colas y APIs).

Índice

1. Descripción del proyecto
2. Requerimientos
3. Modelo de datos
4. Metodología
5. Tecnologías y por qué
6. Cronología de versiones
7. Registro de decisiones
8. Cambios de rumbo y migraciones
9. Flujo de trabajo diario
10. Pruebas y verificación
11. Proceso de release
12. Lecciones aprendidas y riesgos
13. Infraestructura desplegada
14. Pendientes

---

1. Descripción del proyecto

1.1 Contexto

El proyecto implementa un **sistema de manejo de vuelos, pasajeros y equipaje para el Aeropuerto Internacional de la Ciudad de México (AICM)**. El aeropuerto opera varias terminales, decenas de puertas de embarque y un flujo continuo de pasajeros y maletas que deben coordinarse en tiempo real. Un pasajero necesita saber **por qué puerta sale su vuelo, en qué banda llega su equipaje y a qué hora es su conexión**; el personal necesita ver el estado completo del aeropuerto en un solo panel.

El sistema cubre ese flujo de punta a punta: catálogos, vuelos, itinerarios con escalas, reservas, equipaje con conexiones, notificaciones automáticas, panel gráfico en tiempo real, reportería, auditoría y un agente de IA que responde con datos y documentación.

1.2 Enunciado del proyecto

> Sistema de manejo de vuelos, pasajeros y equipaje para el Aeropuerto Internacional de la Ciudad de México. Se debe indicar por medio de notificaciones a los pasajeros en qué puerta les toca su escala, en qué banda está su equipaje y a qué hora sale su vuelo. Todos los vuelos inician como `PROGRAMADOS`, pero pueden sufrir cambios en sus horarios o ser cancelados definitivamente. No puede haber más de un vuelo por puerta al mismo tiempo, y el equipaje a veces viene desde un origen y, sin que el pasajero tome de nuevo el equipaje, debe registrarse para qué destino va. Se debe tener control de todo en todo momento a través de un panel que se pueda ver en tiempo real y que sea gráfico. Como requisito mínimo, se tendrán que maquetar al menos 25 puertas de embarque, calculando los vuelos, la cantidad de pasajeros y la cantidad de equipaje que se mueve en el aeropuerto.

1.3 Alcance implementado

Requisito del enunciado	Cómo se resolvió	Evidencia
Notificar puerta de escala	Aviso `PUERTA_ASIGNADA` / `CAMBIO_PUERTA` al crear o cambiar la puerta de salida de una escala.	`services/escala.service.js`, `npm run test:escalas`
Notificar banda de equipaje	Aviso `BANDA_EQUIPAJE` al asignar banda y al cambiar el estado del equipaje.	`services/equipaje.service.js`
Notificar hora de salida	Aviso inicial de reserva, aviso `CAMBIO_HORARIO` al reprogramar y `VUELO_RETRASADO` al retrasar.	`services/pasajero_vuelo.service.js`, `services/vuelo.service.js`
Vuelos inician `PROGRAMADO`	El alta fuerza `estado = 'PROGRAMADO'`; la máquina de estados controla el resto.	`services/vuelo.service.js`
Cambios de horario y cancelación	Reprogramación con validación de puerta y cancelación con motivo obligatorio.	`services/vuelo.service.js`
Un vuelo por puerta a la vez	Validación en la aplicación **y** restricción `EXCLUDE USING gist` en PostgreSQL.	`db/postgres/schema.sql`
Equipaje en conexión sin retiro	Tramos `equipaje_vuelo` con validación de continuidad de ruta y destino final.	`services/equipaje.service.js`
Panel gráfico en tiempo real	Frontend React + mapa de puertas + Socket.IO (`panel:actualizar`) + respaldo por sondeo.	`frontend/`, `services/panel.service.js`
Al menos 25 puertas	25 puertas sembradas por zona; los vuelos del día se distribuyen entre ellas con sus pasajeros y equipaje.	`db/postgres/seed.sql`, `npm run test:panel`

1.4 Actores y roles

Rol	Quién es	Qué puede hacer
`ADMINISTRADOR`	Personal de sistemas del aeropuerto.	Todo: usuarios, roles, catálogos, vuelos, equipaje, auditoría, reportes y agente.
`SECRETARIO`	Personal operativo de mostrador y sala.	Vuelos, pasajeros, reservas, escalas, equipaje, notificaciones, panel y reportes. No administra usuarios ni auditoría.
`PASAJERO`	Viajero con una cuenta vinculada a su ficha.	Consultar **solo sus** vuelos, escalas, equipaje y notificaciones; usar el agente con alcance propio.

1.5 Fuera del alcance

No forman parte de la entrega: venta de boletos o pasarela de pagos, control migratorio o aduanas, integración con sistemas reales de aerolíneas o del AICM, aplicación móvil nativa, facturación y gestión de tripulaciones. El sistema es una maqueta funcional y desplegada del dominio solicitado.

---

2. Requerimientos

Los requerimientos se agrupan en **negocio** (para qué existe el sistema), **de usuario** (qué necesita cada actor), **funcionales** (qué hace), **no funcionales** (cómo debe comportarse) y **reglas de negocio** (invariantes que nunca se pueden violar). Cada uno tiene un identificador estable para trazabilidad con el código y las pruebas.

2.1 Requerimientos de negocio

ID	Objetivo	Indicador de cumplimiento
OBJ-01	Operar y visualizar al menos **25 puertas** con sus vuelos, pasajeros y equipaje.	Panel con 25 puertas; prueba `test:panel`.
OBJ-02	Dar certidumbre al pasajero sobre **puerta de escala, banda de equipaje y hora de salida**.	Notificaciones automáticas con esos tres datos; pruebas `test:escalas`, `test:rabbitmq`.
OBJ-03	Evitar que dos vuelos ocupen la misma puerta en horarios solapados.	Restricción de base de datos y rechazo `409` en la API.
OBJ-04	Permitir conexiones **sin que el pasajero retire el equipaje**.	Tramos encadenados por aeropuerto de continuidad.
OBJ-05	Tener control y trazabilidad de toda la operación en tiempo real.	Panel Socket.IO, historial de vuelos y equipaje, auditoría.
OBJ-06	Facilitar la toma de decisiones con reportes y visualización.	16 reportes, Excel/PDF y 11 vistas para Power BI.
OBJ-07	Entregar el sistema **desplegado y revisable** sin costo mensual.	Frontend en Vercel, API en Render, base en Neon, broker en CloudAMQP.
OBJ-08	Ofrecer asistencia en lenguaje natural con fuentes verificables.	Agente con RAG y citas; prueba `test:agente`.

2.2 Requerimientos de usuario

ID	Historia de usuario	Criterio de aceptación	RF
HU-01	Como **administrador** quiero gestionar usuarios y roles para controlar quién entra al sistema.	Solo el administrador crea, edita o desactiva cuentas; el resto recibe 403.	RF-01, RF-02

ID	Historia de usuario	Criterio de aceptación	RF
HU-02	Como **secretario** quiero registrar un vuelo y asignarle puerta para publicar la operación del día.	El vuelo nace `PROGRAMADO` y la puerta no tiene conflicto de horario.	RF-03, RF-04, RF-07
HU-03	Como **secretario** quiero reprogramar o cancelar un vuelo para avisar a los pasajeros.	La reprogramación valida la puerta; la cancelación exige motivo y notifica.	RF-06, RF-08
HU-04	Como **secretario** quiero reservar pasajeros sin sobrevender el vuelo.	El cupo se valida en transacción y el exceso devuelve 400.	RF-12
HU-05	Como **secretario** quiero registrar el equipaje en conexión sin que el pasajero lo retire.	El tramo nuevo continúa desde el destino del anterior.	RF-14, RF-15
HU-06	Como **pasajero** quiero recibir la puerta de mi escala, la banda de mi equipaje y la hora de mi vuelo.	Los tres avisos llegan a `notificaciones` y a la bandeja de correo.	RF-17, RF-18
HU-07	Como **pasajero** quiero consultar solo mis vuelos, escalas y equipaje.	Una consulta a datos de otro pasajero no devuelve registros.	RF-27, RN-20
HU-08	Como **supervisor** quiero ver las 25 puertas, los vuelos y los KPIs en tiempo real.	El panel se actualiza sin recargar cuando cambia un dato.	RF-19 a RF-21
HU-09	Como **analista** quiero reportes filtrables y exportables para revisar la operación.	Cada reporte descarga Excel y PDF con los filtros aplicados.	RF-22 a RF-24
HU-10	Como **administrador** quiero auditar los cambios para reconstruir lo ocurrido.	Cada alta, cambio o baja deja fila en `auditoria` con usuario y fecha.	RF-25, RF-26
HU-11	Como **usuario** quiero preguntar en lenguaje natural y ver las fuentes de la respuesta.	El agente responde con datos reales y cita documentos del corpus.	RF-27

2.3 Requerimientos funcionales

ID	Requerimiento	Implementación	Verificación
RF-01	Iniciar sesión con usuario y contraseña y obtener un JWT.	`POST /api/auth/login`, `GET /api/auth/me`	`test:api`
RF-02	Aplicar permisos por rol en cada ruta.	Middlewares JWT y roles	`test:api`
RF-03	Gestionar vuelos (alta, consulta, edición y eliminación).	`/api/vuelos`	`test:api`
RF-04	Crear todo vuelo en estado `PROGRAMADO`.	`crearVuelo` fuerza el estado	`test:api`
RF-05	Cambiar el estado del vuelo solo por transiciones válidas.	`/api/vuelos/:id/estado`	`test:api`
RF-06	Reprogramar fecha y hora de salida o llegada.	`/api/vuelos/:id/reprogramar`	`test:api`
RF-07	Asignar o cambiar la puerta de un vuelo.	`/api/vuelos/:id/puerta`	`test:api`
RF-08	Cancelar un vuelo indicando el motivo.	`/api/vuelos/:id/cancelar`	`test:api`
RF-09	Administrar el catálogo de 25 puertas y sus estados.	`/api/puertas`	`test:panel`
RF-10	Administrar el catálogo de 10 bandas de equipaje.	`/api/bandas-equipaje`	`test:panel`
RF-11	Administrar aeropuertos, pasajeros e itinerarios.	`/api/aeropuertos`, `/api/pasajeros`, `/api/itinerarios`	`test:api`
RF-12	Reservar un pasajero en un vuelo con control de cupo y asiento.	`/api/pasajero-vuelo`	`test:api`

ID	Requerimiento	Implementación	Verificación
RF-13	Registrar escalas o conexiones de un itinerario.	`/api/escalas`	`test:escalas`
RF-14	Registrar equipaje con origen y destino final.	`/api/equipajes`	`test:api`
RF-15	Asignar el equipaje a los vuelos de su ruta sin retirarlo.	`/api/equipaje-vuelo`, `POST /api/equipajes/:id/tramos`	`test:api`
RF-16	Asignar banda y cambiar el estado del equipaje.	`PATCH /api/equipajes/:id/banda`, `PATCH /api/equipajes/:id/estado`	`test:api`
RF-17	Generar notificaciones automáticas por evento de vuelo, escala o equipaje.	`services/notificacion.service.js` y hooks	`test:api`, `test:escalas`
RF-18	Entregar las notificaciones por correo de forma asíncrona, con reintentos y traza.	RabbitMQ + worker + `entregas_notificacion`	`test:rabbitmq`
RF-19	Exponer un panel operativo con KPIs, puertas, bandas, vuelos y avisos.	`GET /api/panel`	`test:panel`
RF-20	Actualizar el panel en tiempo real sin recargar.	Socket.IO `panel:actualizar`	`test:panel`
RF-21	Ofrecer una interfaz gráfica con login, mapa de puertas, vuelos y reportes.	`frontend/` React + Vite	`npm run build`, E2E manual
RF-22	Consultar 16 reportes con filtros y agregaciones.	`/api/reportes/:clave`	`test:reportes`
RF-23	Exportar cualquier reporte a Excel y PDF.	`/api/reportes/:clave/exportar?formato=`	`test:reportes`
RF-24	Exponer vistas de solo lectura para Power BI.	`db/postgres/views_bi.sql`	`npm run db:views`
RF-25	Registrar historial de cambios de vuelo y movimientos de equipaje.	`historial_vuelo`, `historial_equipaje`	`test:api`
RF-26	Auditar las operaciones relevantes con usuario, tabla y fecha.	`auditoria` + hooks Sequelize	`test:api`
RF-27	Responder preguntas en español con datos operativos y citas documentales.	`POST /api/agente/consultas`	`test:agente`
RF-28	Informar el estado de salud de la API, la base y la mensajería.	`GET /health`	Despliegue

2.4 Requerimientos no funcionales

ID	Categoría	Requerimiento	Cómo se cumple
RNF-01	Consistencia	Las operaciones que tocan varias tablas deben ser atómicas.	Transacciones Sequelize y `FOR UPDATE` en reservas y equipaje.
RNF-02	Integridad	No debe existir un vuelo con dos puertas solapadas ni referencias huérfanas.	`EXCLUDE USING gist`, claves foráneas, `CHECK` y `UNIQUE`.
RNF-03	Seguridad	Las rutas deben exigir autenticación y respetar el rol.	JWT (8 h), bcrypt, middlewares de rol, CORS configurable.
RNF-04	Seguridad	El agente no debe exponer datos de otros pasajeros ni modificar datos.	Filtro por pasajero en SQL; solo consultas.
RNF-05	Seguridad	El endpoint del agente debe resistir abuso.	Límite de 30 consultas/minuto por usuario.

ID	Categoría	Requerimiento	Cómo se cumple
RNF-06	Rendimiento	El panel debe cargar en una sola petición y refrescarse por evento.	`GET /api/panel`, Socket.IO y sondeo de respaldo cada 30 s.
RNF-07	Concurrencia	Dos solicitudes simultáneas no deben sobrevender ni duplicar reservas.	Bloqueos `FOR UPDATE` y restricciones únicas.
RNF-08	Disponibilidad	Debe existir un chequeo de salud verificable.	`/health` valida base y mensajería; Render lo usa como health check.
RNF-09	Recuperación	Un fallo temporal de correo no debe perder el aviso.	3 reintentos con TTL y cola de fallidas en RabbitMQ.
RNF-10	Portabilidad	El sistema debe correr en local y en la nube sin cambios de código.	Docker Compose, variables de entorno y despliegue en Render/Vercel/Neon.
RNF-11	Mantenibilidad	El código debe separar rutas, controladores, servicios, modelos y validación.	Arquitectura por capas, Zod y `asyncHandler`.
RNF-12	Usabilidad	La interfaz debe ser legible y accesible.	Modo claro/oscuro, estado con icono y etiqueta, foco visible, responsive a 390 px.
RNF-13	Compatibilidad	Debe funcionar en navegadores modernos y PostgreSQL 14+.	React 18, Vite 5, PostgreSQL 16 en local y Neon en producción.
RNF-14	Observabilidad	Debe poder reconstruirse quién cambió qué y cuándo.	Historiales, auditoría, logs del worker y `interacciones_agente`.
RNF-15	Localización	El idioma del sistema y de los mensajes es español; la zona horaria es de México.	`TZ=America/Mexico_City`, textos en español.
RNF-16	Costo	La operación de la demo no debe generar costo mensual.	Planes gratuitos de Render, Vercel, Neon, CloudAMQP y Ethereal.
RNF-17	Documentación	Debe existir documentación de usuario, administración y desarrollo.	Los tres manuales, README, bitácora y guías de despliegue.

2.5 Reglas de negocio

ID	Regla	Dónde se aplica
RN-01	Todo vuelo nuevo nace en estado `PROGRAMADO`.	`services/vuelo.service.js` (`crearVuelo`).
RN-02	Solo se permiten las transiciones de la máquina de estados del vuelo.	`TRANSICIONES` en `vuelo.service.js`.
RN-03	Un vuelo `FINALIZADO` o `CANCELADO` no se puede modificar.	`actualizarVuelo`.
RN-04	Cancelar exige un motivo no vacío.	`actualizarVuelo` / `cancelarVuelo`.
RN-05	Un vuelo cancelado libera su puerta.	El `EXCLUDE` excluye `CANCELADO`; la API no valida conflicto al cancelar.
RN-06	Una puerta no puede tener dos vuelos activos con horarios solapados.	API (`verificarPuertaDisponible`) y `no_solape_puerta` en PostgreSQL.
RN-07	La puerta asignada debe existir y estar `DISPONIBLE`.	`verificarPuertaDisponible`.
RN-08	La llegada debe ser posterior a la salida y el origen distinto del destino.	`validarRangoHorario` y `CHECK` en `vuelos`.
RN-09	No se puede reducir la capacidad por debajo de los pasajeros activos.	`actualizarVuelo`.
RN-10	No hay sobreventa: la reserva revalida el cupo con el vuelo bloqueado.	`services/pasajero_vuelo.service.js`.

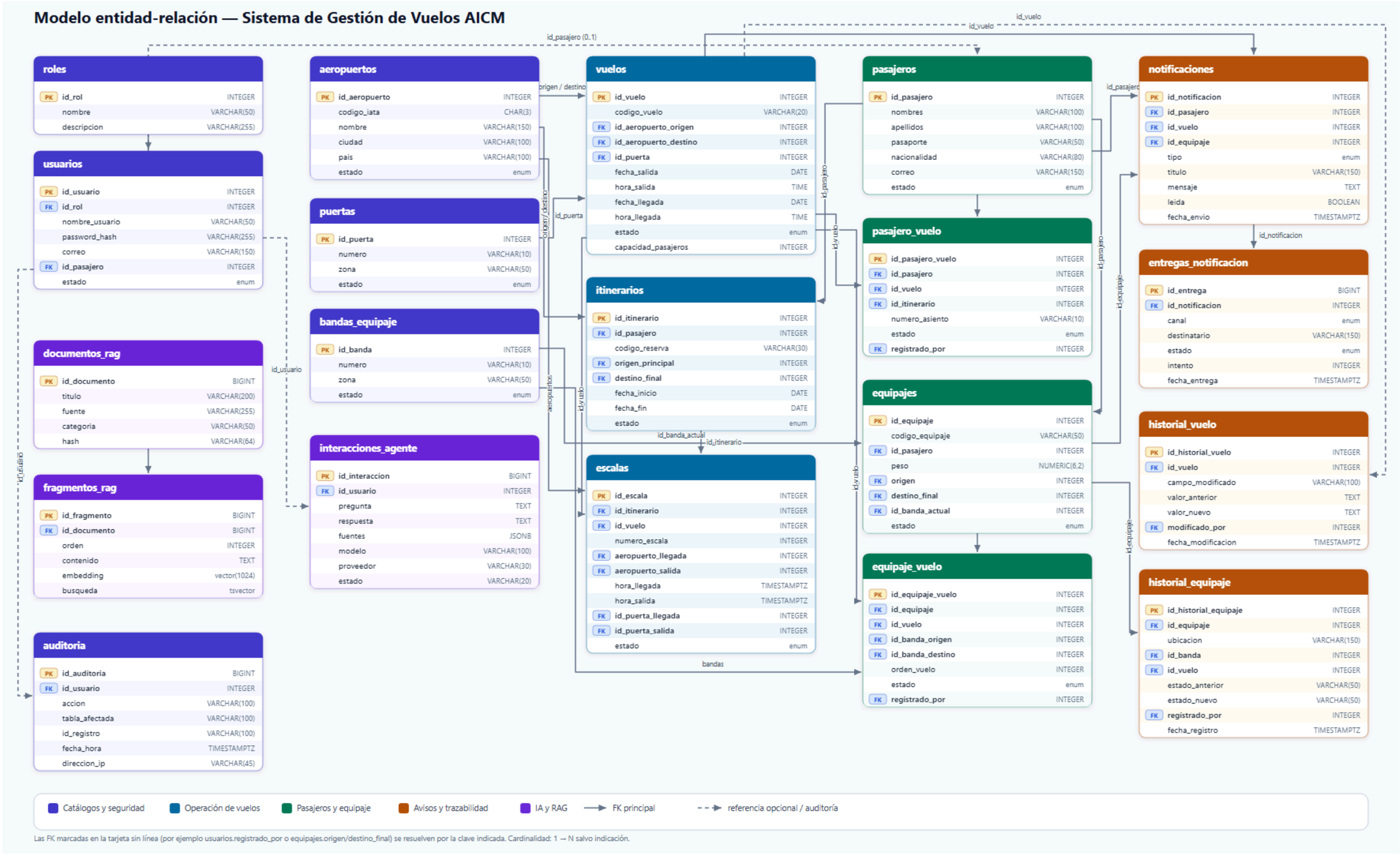
ID	Regla	Dónde se aplica
RN-11	Una reserva cancelada libera su lugar; reactivarla vuelve a validar cupo y estado.	`pasajero_vuelo.service.js`.
RN-12	El primer tramo del equipaje debe iniciar en su aeropuerto de origen.	`services/equipaje.service.js`.
RN-13	Cada tramo debe continuar desde el destino del tramo anterior.	`equipaje.service.js` (continuidad de ruta).
RN-14	Los tramos de equipaje son inmutables: no hay `PUT` ni `DELETE`; solo se agregan por el flujo de negocio.	`routes/equipaje_vuelo.routes.js`.
RN-15	El equipaje solo cambia de estado por transiciones válidas; `ENTREGADO` es terminal.	`TRANSICIONES_EQUIPAJE`.
RN-16	Solo se asigna equipaje a vuelos en estado operativo (`PROGRAMADO`, `RETRASADO`, `ABORDANDO`, `ABORDADO`).	`ESTADOS_QUE_PERMITEN_REGISTRO`.
RN-17	La banda de equipaje debe existir y estar `DISPONIBLE`.	`validarBanda`.
RN-18	Una escala debe pertenecer a un itinerario válido y tener aeropuertos, horarios y puertas operativas coherentes.	`services/escala.service.js`.
RN-19	`registrado_por`, `modificado_por` y el usuario de auditoría se derivan del token, nunca del cuerpo.	Controladores y hooks.
RN-20	Un pasajero solo puede consultar sus propios vuelos, escalas, equipaje y notificaciones.	Filtro en SQL de `agente.tools.js` y rutas de pasajero.
RN-21	Toda operación relevante se registra en historial y auditoría.	Hooks de Sequelize y servicios de historial.
RN-22	El agente de IA es de solo consulta: no crea, modifica ni elimina datos.	`services/agente.tools.js`.

2.6 Trazabilidad

Módulo	Requerimientos	Prueba
Autenticación y permisos	RF-01, RF-02, RNF-03	`npm run test:api`
Vuelos y puertas	RF-03 a RF-09, RN-01 a RN-09	`npm run test:api`
Reservas y escalas	RF-11 a RF-13, RN-10, RN-11, RN-18	`npm run test:api`, `npm run test:escalas`
Equipaje	RF-14 a RF-16, RN-12 a RN-17	`npm run test:api`
Notificaciones y mensajería	RF-17, RF-18, RNF-09	`npm run test:rabbitmq`, `npm run test:escalas`
Panel y tiempo real	RF-19 a RF-21, RNF-06	`npm run test:panel`
Reportería y BI	RF-22 a RF-24	`npm run test:reportes`
Historial y auditoría	RF-25, RF-26, RNF-14	`npm run test:api`
Agente y RAG	RF-27, RNF-04, RNF-05	`npm run test:agente`
Operación desplegada	RF-28, OBJ-07	`/health` y checklist de despliegue

3. Modelo de datos

3.1 Diagrama entidad-relación



También está disponible en vectorial: `img/diagrama-er.svg`. El esquema ejecutable es `db/postgres/schema.sql`; los datos de demostración, `db/postgres/seed.sql`.

El modelo tiene **20 tablas** agrupadas en cinco dominios:

Dominio	Tablas
Catálogos y seguridad	`roles`, `usuarios`, `aeropuertos`, `puertas`, `bandas_equipaje`
Operación de vuelos	`vuelos`, `itinerarios`, `escalas`
Pasajeros y equipaje	`pasajeros`, `pasajero_vuelo`, `equipajes`, `equipaje_vuelo`
Avisos y trazabilidad	`notificaciones`, `entregas_notificacion`, `historial_vuelo`, `historial_equipaje`, `auditoria`
IA y RAG	`documentos_rag`, `fragmentos_rag`, `interacciones_agente`

3.2 Diccionario de tablas

Tabla	Propósito	Clave primaria	Claves foráneas principales
`roles`	Catálogo de roles del sistema.	`id_rol`	—
`usuarios`	Cuentas, credenciales y vínculo opcional con un pasajero.	`id_usuario`	`id_rol` → `roles`; `id_pasajero` → `pasajeros`
`aeropuertos`	Catálogo de aeropuertos por código IATA.	`id_aeropuerto`	—
`pasajeros`	Ficha del pasajero (pasaporte único).	`id_pasajero`	—
`puertas`	25 puertas de embarque con zona y estado.	`id_puerta`	—
`bandas_equipaje`	10 bandas de reclamo con zona y estado.	`id_banda`	—
`vuelos`	Vuelo, horario, puerta, capacidad y estado.	`id_vuelo`	origen y destino → `aeropuertos`; `id_puerta` → `puertas`
`itinerarios`	Viaje completo de un pasajero con su reserva.	`id_itinerario`	`id_pasajero`; origen y destino → `aeropuertos`
`pasajero_vuelo`	Reserva de un pasajero en un vuelo (tramo).	`id_pasajero_vuelo`	`id_pasajero`, `id_vuelo`, `id_itinerario`, `registrado_por`
`escalas`	Conexiones del itinerario y sus puertas.	`id_escala`	`id_itinerario`, `id_vuelo`, aeropuertos y puertas de llegada/salida
`equipajes`	Maleta con origen, destino final, banda y estado.	`id_equipaje`	`id_pasajero`; origen y destino → `aeropuertos`; `id_banda_actual`
`equipaje_vuelo`	Tramos de la maleta en cada vuelo.	`id_equipaje_vuelo`	`id_equipaje`, `id_vuelo`, bandas de origen/destino, `registrado_por`
`notificaciones`	Avisos generados para el pasajero.	`id_notificacion`	`id_pasajero`, `id_vuelo`, `id_equipaje`
`entregas_notificacion`	Traza de entrega por canal (correo, push, SMS).	`id_entrega`	`id_notificacion`
`historial_vuelo`	Cambios de campos y estado de un vuelo.	`id_historial_vuelo`	`id_vuelo`, `modificado_por` → `usuarios`
`historial_equipaje`	Movimientos y cambios de estado del equipaje.	`id_historial_equipaje`	`id_equipaje`, `id_banda`, `id_vuelo`, `registrado_por`

Tabla	Propósito	Clave primaria	Claves foráneas principales
`auditoria`	Registro de operaciones por usuario y tabla.	`id_auditoria`	`id_usuario` → `usuarios`
`documentos_rag`	Documentos del corpus del agente.	`id_documento`	—
`fragmentos_rag`	Fragmentos con `tsvector` y embedding opcional.	`id_fragmento`	`id_documento`
`interacciones_agente`	Preguntas, respuestas, fuentes y métricas del agente.	`id_interaccion`	`id_usuario` → `usuarios`

3.3 Tipos enumerados

Tipo	Valores
`estado_activo`	`ACTIVO`, `INACTIVO`
`estado_puerta_banda`	`DISPONIBLE`, `MANTENIMIENTO`, `INACTIVA`
`estado_vuelo`	`PROGRAMADO`, `ABORDANDO`, `ABORDADO`, `EN_VUELO`, `ATERRIZADO`, `FINALIZADO`, `RETRASADO`, `CANCELADO`
`estado_equipaje`	`REGISTRADO`, `EN_TRANSITO`, `EN_BANDA`, `CARGADO`, `TRANSFERIDO`, `ENTREGADO`, `EXTRAVIADO`, `DANADO`
`estado_equipaje_vuelo`	`REGISTRADO`, `CARGADO`, `EN_VUELO`, `RECIBIDO`, `TRANSFERIDO`, `ENTREGADO`
`estado_escala`	`PENDIENTE`, `EN_TRANSITO`, `COMPLETADA`, `CANCELADA`
`estado_itinerario`	`PROGRAMADO`, `EN_CURSO`, `COMPLETADO`, `CANCELADO`
`estado_pasajero_vuelo`	`RESERVADO`, `CHECK_IN`, `ABORDADO`, `NO_ABORDADO`, `CANCELADO`
`tipo_notificacion`	`CAMBIO_PUERTA`, `CAMBIO_HORARIO`, `VUELO_CANCELADO`, `VUELO_RETRASADO`, `PUERTA_ASIGNADA`, `BANDA_EQUIPAJE`, `ESCALA`, `INFORMACION_GENERAL`
`canal_notificacion`	`CORREO`, `PUSH`, `SMS`
`estado_entrega`	`PENDIENTE`, `ENVIADA`, `SIMULADA`, `FALLIDA`

3.4 Restricciones e integridad

Mecanismo	Dónde	Qué garantiza
`EXCLUDE USING gist`	`vuelos.no_solape_puerta`	Ningún vuelo no cancelado comparte puerta con intervalos solapados, incluso con solicitudes concurrentes.
`CHECK`	`vuelos.capacidad_pasajeros >= 0`, fechas de llegada > salida, `equipajes.peso >= 0`	Rangos y coherencia temporal.
`UNIQUE`	Pasaporte, código IATA, número de puerta/banda, código de reserva, `(pasajero, vuelo)`, `(equipaje, vuelo)`, `(itinerario, número de escala)`, `(documento, orden)`, `(notificación, canal)`	Evita duplicados de negocio.
Claves foráneas	Todas las relaciones del diagrama	Integridad referencial con `RESTRICT`, `SET NULL` o `CASCADE` según el caso.
`DEFERRABLE`	`usuarios.id_pasajero`	Permite sembrar usuarios antes que pasajeros.

Mecanismo	Dónde	Qué garantiza
Triggers	`set_updated_at()` en 12 tablas	Actualiza `updated_at` en cada modificación.
Extensiones	`btree_gist`, `vector`	Restricción de exclusión y búsqueda vectorial del RAG.
Índices	Fechas, estados, códigos y todas las FK	Consultas del panel, reportes y agente.

3.5 Vistas para Power BI

`db/postgres/views_bi.sql` define 11 vistas de solo lectura: `bi_vuelos`, `bi_pasajeros_vuelo`, `bi_equipajes`, `bi_equipaje_tramos`, `bi_notificaciones`, `bi_entregas_notificacion`, `bi_historial_vuelo`, `bi_historial_equipaje`, `bi_auditoria`, `bi_kpis_diarios` y `bi_interacciones_agente`. Se recrean con `npm run db:views` sin borrar datos.

---

4. Metodología

El proyecto se construyó con **desarrollo guiado por especificaciones** y apoyo de Claude Code. La evidencia está en el repositorio:

- `CLAUDE.md`: instrucciones para Claude Code (comandos, convenciones, arquitectura, flujo de ramas y reglas de verificación).
- `AGENTS.md`: resumen para otros agentes (Codex, OpenCode, etc.).
- `docs/BITACORA.md`: registro append-only de cada sesión, con objetivo, rama, archivos, verificación real, versión, commits, pendientes y riesgos.
- `docs/ANALISIS_FALTANTES*.md`: análisis de brechas frente a la rúbrica.
- `manual/`: estos tres manuales.

4.1 Principios

1. **Especificar antes de codificar:** el requisito se escribe y se contrasta con la rúbrica.
2. **Verificar con evidencia:** no se declara terminado lo que no se ejecutó.
3. **Una rama y un PR por módulo.**
4. **Commit descriptivo:** qué se hizo, por qué y cómo se verificó.
5. **No inflar el avance:** la bitácora refleja solo el diff real.
6. **Los commits los ejecuta el estudiante** (regla del curso).
7. **SemVer:** MAJOR rompe, MINOR agrega, PATCH corrige.

---

5. Tecnologías y por qué

Tecnología	Uso	Por qué
Node.js + Express 5	Backend	Ecosistema maduro y despliegue sencillo.
PostgreSQL	Base	`EXCLUDE USING gist`, tipos, transacciones, pgvector.
Sequelize 6	ORM	Modelos, asociaciones, transacciones y hooks.
SQL directo	Reportes, RAG	Agregaciones y pgvector que el ORM no cubre.
Zod 4	Validación	Contratos explícitos.
JWT + bcryptjs	Auth	API sin estado y Socket.IO.
React + Vite	Frontend	Componentes y build estático.
Socket.IO	Tiempo real	Señal `panel:actualizar`.
RabbitMQ + amqplib	Mensajería	Asincronía, reintentos y traza.
CloudAMQP	Broker	Plan gratuito administrado.
Nodemailer + Ethereal	Correo	SMTP real de pruebas, gratis.
ExcelJS + PDFKit	Reportes	Exportación sin servicios externos.
pgvector	RAG	Búsqueda vectorial en PostgreSQL.
@anthropic-ai/sdk	Agente	Tool use y modelo Claude.
Docker Compose	Local	Entorno reproducible.
Render	API + worker	Plan gratuito con health check.
Vercel	Frontend	CDN y despliegue desde Git.
Neon	Base en la nube	Plan gratuito y pgvector.

Detalle de arquitectura y operación: `02-MANUAL_ADMINISTRADOR_PROGRAMADOR.md`.

---

6. Cronología de versiones

Versión	Qué se hizo	Por qué
v1.0.0	Backend base; migración MySQL → PostgreSQL.	`EXCLUDE` para puertas, tipos y transacciones; PostgreSQL en Neon.
v1.1.0	Roles, validaciones, máquina de estados, conflicto de puertas, notificaciones, historial y auditoría.	Reglas del dominio y rúbrica.
v1.1.1	Cupo concurrente, tramos inmutables, estados de equipaje y `/health`.	Endurecer invariantes.
v1.2.0	16 reportes, Excel/PDF y 9 vistas BI.	Reportería y Power BI.
v1.3.0	Panel operativo, Socket.IO y CORS.	Tiempo real y permisos.
v1.4.0	Frontend React/Vite.	Interfaz para la operación y la defensa.
v1.5.0	RabbitMQ, worker, entregas y reintentos.	Mensajería asíncrona real.

Versión	Qué se hizo	Por qué
v1.5.1	Despliegue, Docker local, supervisor, seed relativo y contraseñas.	El enunciado no revisa sistemas no desplegados; la demo necesitaba datos del día.
v1.6.0	Agente IA + RAG, herramientas, logs, rate limit y chat.	Requisito de agente y RAG con citas.
v1.6.1	Aviso inicial de reserva, aviso de puerta de escala, servicio de escalas y corrección del troceado RAG.	Requisitos de dominio y calidad del RAG.
v1.7.0 (pendiente)	`.pbix` de Power BI y cierre.	Último bloque de la rúbrica.

6.1 Estado de GitHub

PR	Versión	Estado
#10	v1.5.0 (RabbitMQ)	Fusionado.
#11	v1.5.1 (despliegue)	Fusionado por Demy.
#12	v1.6.0 (agente + RAG)	Abierto, `MERGEABLE`, pendiente de revisión.
#13	v1.6.1 (dominio y RAG)	Abierto, apilado sobre #12.

Los tags `v1.5.1`, `v1.6.0` y `v1.6.1` existen en el fork. En `main` se crean al fusionar.

---

7. Registro de decisiones

#	Decisión	Alternativas	Por qué	Consecuencia
1	PostgreSQL	MySQL, SQLite	`EXCLUDE USING gist`, tipos y transacciones; SQLite está prohibido.	Migración v1.0.0; Neon en producción.
2	Node + JavaScript	Python/FastAPI, TypeScript	Código existente y ecosistema.	Se añadió complejidad; conviene confirmarlo con el catedrático.
3	Arquitectura por capas	Un solo archivo	Mantenibilidad y rúbrica.	Más archivos con responsabilidades claras.
4	JWT + bcrypt	Sesiones	API sin estado, Socket.IO y frontend separado.	Token en el cliente; expira en 8 h.
5	Zod	Validación manual	Contratos explícitos y errores consistentes.	Esquemas que mantener.
6	Sequelize + SQL directo	Solo ORM o solo SQL	ORM para CRUD/hooks; SQL para reportes y RAG.	Dos estilos documentados.
7	25 puertas y 10 bandas	Menos entidades	Escala realista.	El seed debe respetar la exclusión.
8	RabbitMQ	Envío directo	Asincronía y reintentos.	Requiere broker y worker.
9	CloudAMQP	Broker propio	Gratuito y administrado.	Límites del plan gratuito.
10	Worker en el mismo servicio (Render)	Worker separado	El plan free no incluye Background Workers.	El worker se pausa cuando Render duerme.
11	Neon	PostgreSQL de Render	Free, pgvector y pooled.	`DB_SSL=true`.

#	Decisión	Alternativas	Por qué	Consecuencia
12	Vercel para el frontend	Servir desde Express	CDN y build estático.	CORS y `VITE_API_URL`.
13	Rechazar Vercel multi-servicio	API + frontend en Vercel	Socket.IO y worker necesitan proceso permanente.	Backend en Render.
14	Ethereal	Gmail, Mailtrap	Gratis e inmediato.	No entrega a buzones reales.
15	Docker en 5433	PostgreSQL nativo	El 5432 estaba ocupado.	Entorno reproducible.
16	Seed relativo	Fechas fijas	La demo mostraba 0 vuelos hoy.	Siempre hay vuelos del día.
17	Rotar contraseñas	Dejarlas públicas	Seguridad.	`db:password` y variables de prueba.
18	Fork para desplegar	Pedir acceso al dueño	La cuenta no era dueña del repo.	Render/Vercel leen el fork.
19	Agente local por defecto	Exigir API key	Demostrar sin costo.	Respuestas deterministas.
20	Claude opcional	Otro proveedor	Tool use y calidad.	Requiere `ANTHROPIC_API_KEY`.
21	RAG híbrido	Solo vectores o solo texto	Funcionar sin clave de embeddings.	Texto por defecto; vectores si hay Voyage.
22	Rate limit del agente	Sin límite	Evitar abuso y costos.	30/min por usuario, en memoria.
23	Servicio de escalas + avisos	Seguir en el controlador	Código limpio y requisito de puerta de escala.	Refactor y notificaciones.
24	Credenciales de prueba por entorno	Hardcodear	Las contraseñas cambian.	`ADMIN_PASSWORD`, etc.
25	SemVer + bitácora + PRs	Commits sueltos	Trazabilidad y rúbrica.	PR apilados y rebases.

---

8. Cambios de rumbo y migraciones

8.1 MySQL → PostgreSQL

El dump original estaba en MySQL. Se migró a PostgreSQL por la restricción `EXCLUDE USING gist`, los tipos y las transacciones. El esquema oficial es `db/postgres/schema.sql`; el dump MySQL queda como referencia en `Base de datos/`.

8.2 Fechas fijas → seed relativo

El seed original usaba el 1–6 de octubre de 2026. Con el paso de los días el panel mostraba "Vuelos hoy = 0". Ahora el 2 de octubre se toma como "hoy" y todas las fechas se desplazan a la fecha real de ejecución.

8.3 Worker separado → supervisor en un solo servicio

Render gratuito no ofrece *Background Workers*. `scripts/iniciar-produccion.js` ejecuta API y worker, reinicia el worker si se cae y detiene todo si la API termina. Cuando Render duerme, el worker también.

8.4 Vercel multi-servicio → frontend estático + backend en Render

Vercel sugirió un `vercel.json` con servicios `app` y `frontend`. Se descartó: Socket.IO y el worker requieren un proceso permanente. El frontend se despliega solo con `VITE_API_URL` hacia Render.

8.5 Agente: local por defecto, Claude opcional

No había API key de Claude. El agente funciona en modo local con RAG y herramientas; si se define `ANTHROPIC_API_KEY` y `AGENTE_PROVEEDOR=claude`, usa Claude con tool use. Si Claude falla, vuelve al modo local con un aviso.

8.6 RAG: un fragmento por documento → 13 fragmentos

El troceado solo dividía por `\n\n`; con CRLF cada documento quedaba en un solo fragmento. Se corrigió para aceptar `\r?\n\s*\r?\n`, partir bloques largos y reindexar si cambia el número de fragmentos.

8.7 Repositorio original → fork para desplegar

El repositorio pertenece a Demy. La cuenta de Render/Vercel no podía verlo, así que se creó el fork `jjemd101191/Sistema-gestion-de-vuelos`. El repositorio original sigue siendo el entregable; el fork se sincroniza con `git push fork`.

8.8 Contraseñas del seed → rotación

`admin` y `secretario` tenían contraseñas públicas. Se rotaron en producción con `npm run db:password`; las pruebas aceptan las nuevas por entorno.

---

9. Flujo de trabajo diario

9.1 Antes de tocar código

1. `git status -sb` y `git log --oneline -10`.
2. Leer la última entrada de `docs/BITACORA.md`.
3. Revisar la versión en `package.json` y el último tag.
4. Identificar la rama y si hay PR abierto (`gh pr list`).
5. Escribir el objetivo de la sesión en una frase.

9.2 Implementación

1. Crear rama `feature/vX.Y.Z-tema` desde `main` (o desde la rama de la release anterior si está apilada).
2. Implementar por capas: ruta → controlador → servicio → modelo/consulta.
3. Validar entradas con Zod y usar transacciones donde corresponda.
4. Respetar el alcance por rol.
5. Actualizar esquema, migración y seed si cambia el modelo.
6. Actualizar README, `.env.example`, manuales y bitácora.

9.3 Formato de commit

```
<tipo>(<alcance>): <resumen ≤72 caracteres>

Qué se hizo:
- ...

Por qué:
- ...

Verificación:
- comando → resultado
```

Tipos: `feat`, `fix`, `docs`, `refactor`, `test`, `chore`, `perf`, `ci`.

9.4 PR

- Un PR por módulo.
- Título con la versión y el tema.
- Descripción con resumen, verificación y notas de despliegue.
- Si hay PRs apilados, indicar la dependencia.
- La fusión a `main` es la release.
- El dueño del repositorio revisa y fusiona.

9.5 Cierre de sesión

1. `git status` revisado; sin `.env` ni secretos.
2. Pruebas ejecutadas con resultado real.
3. Entrada nueva en `docs/BITACORA.md` (arriba, append-only).
4. Versión decidida y justificada.
5. Commit con qué, por qué y verificación.
6. Tag y PR si es una release.

---

10. Pruebas y verificación

Suite	Comando	Qué verifica
API	`npm run test:api`	CRUD, permisos, estados, cupo, conflictos y auditoría.
Reportería	`npm run test:reportes`	16 reportes, filtros y exportaciones.
Panel	`npm run test:panel`	KPIs, puertas, bandas y Socket.IO.
RabbitMQ	`npm run test:rabbitmq`	API → cola → worker → entrega.
Agente	`npm run test:agente`	RAG, citas, datos operativos, alcance y logs.

Suite	Comando	Qué verifica
Escalas	`npm run test:escalas`	Avisos de reserva y de puerta de escala.
Todas	`npm test`	API + reportes + panel + agente + escalas.

Resultados de referencia: **66 + 29 + 14 + 12 + 11 PASS** y **10 PASS** en RabbitMQ.

> `test:api` no es idempotente: vuelve a ejecutar `db:init` después de correrla.
> No la ejecutes contra producción.

10.1 Pruebas manuales recomendadas antes de una demo

- Login con admin y secretario.
- Panel con 25 puertas y KPIs.
- Un reporte en Excel y otro en PDF.
- Una notificación y su entrega en Ethereal.
- Una pregunta al agente con citas.
- `/health` en producción.

---

11. Proceso de release

1. Rama limpia y pruebas ejecutadas.
2. Actualizar `package.json` (backend y frontend).
3. Actualizar README, manuales y bitácora.
4. Commit `chore(release): vX.Y.Z`.
5. Tag anotado: `git tag -a vX.Y.Z -m "..."`.
6. Push de rama y tag al fork.
7. PR a `main`; revisión y fusión por el dueño.
8. Verificar `/health` y las suites de producción.
9. Crear el tag en `main` después de fusionar.

11.1 SemVer en este proyecto

Cambio	Número	Ejemplo
Incompatible	MAJOR	Renombrar endpoints o cambiar contratos.
Funcionalidad nueva	MINOR	RabbitMQ, agente, reportes.
Corrección compatible	PATCH	Seed relativo, avisos, fixes.

---

12. Lecciones aprendidas y riesgos

1. **El despliegue es un gate:** sin URL pública el sistema no se revisa. Se desplegó temprano.
2. **El plan gratuito condiciona la arquitectura:** el worker comparte servicio con la API.
3. **Las fechas fijas envejecen:** el seed relativo resolvió la demo vacía.
4. **Los permisos de GitHub importan:** el fork fue necesario para desplegar.
5. **Los squash merges obligan a rebasar:** los PR apilados se rebasan al fusionar.
6. **El CRLF afecta el RAG:** el troceado debe aceptar ambos finales de línea.
7. **La verificación debe ser real:** la bitácora registra comandos y resultados.
8. **Los secretos no van al repositorio:** `.env` local y paneles de despliegue.
9. **El modo local del agente es honesto:** no es un LLM generativo; se documenta.
10. **El alcance por rol se aplica en SQL:** no basta con el prompt.

---

13. Infraestructura desplegada

Esta sección concentra **todo lo que se usó para publicar el sistema**: servicios, URLs, credenciales, variables, colas, APIs y comandos de verificación. Complementa a `docs/DESPLIEGUE.md`, que describe el procedimiento paso a paso.

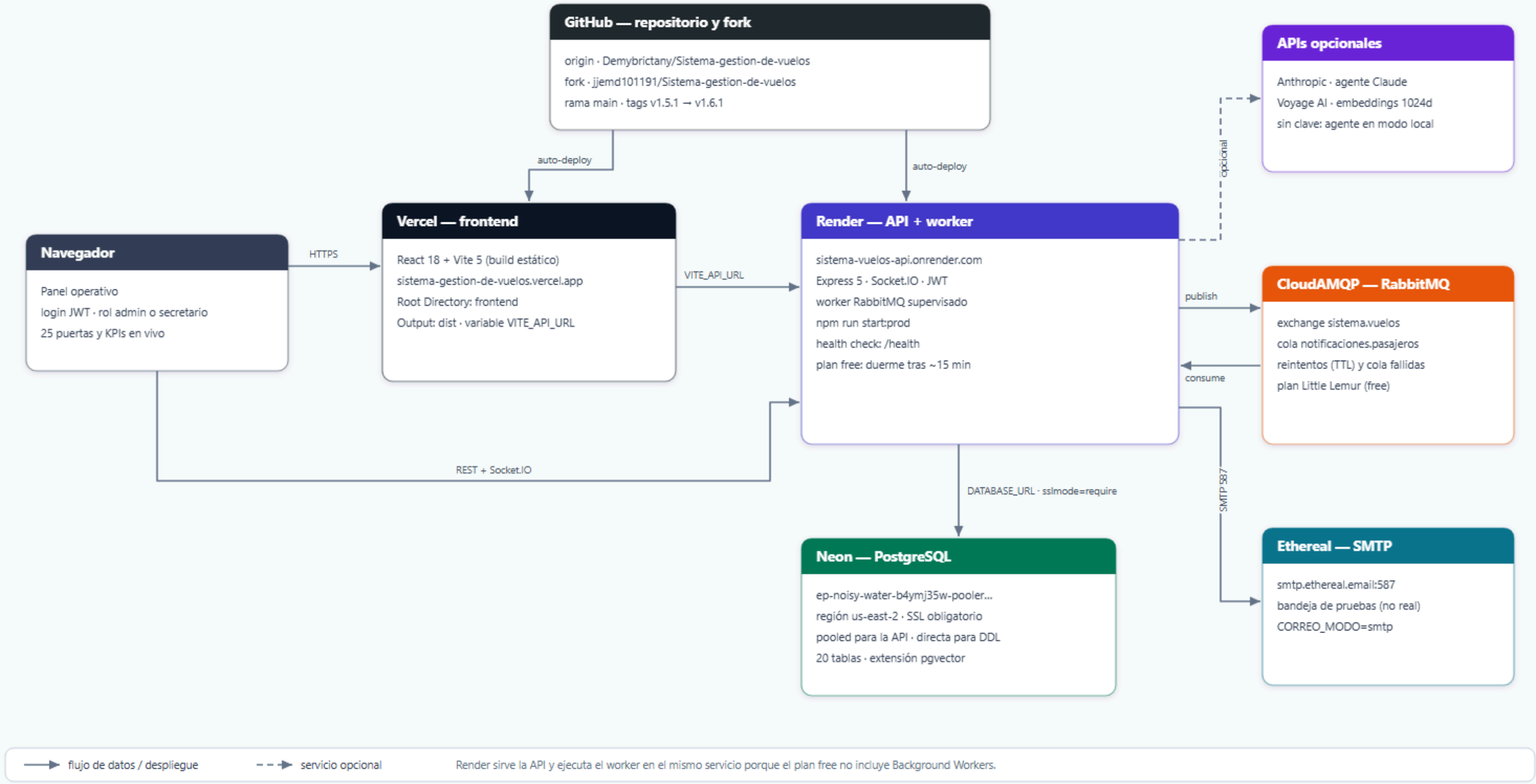
Secuencia que se siguió:

1. Se creó la base en **Neon**, se cargó `schema.sql` con la cadena directa y se ejecutó el seed relativo.
2. Se creó la instancia de **CloudAMQP** y se copió su AMQP URL.
3. Se creó la cuenta de **Ethereal** para el SMTP de pruebas.
4. Se desplegó la API + worker en **Render** con `render.yaml` y las variables marcadas con `sync: false`.
5. Se desplegó el frontend en **Vercel** con `VITE_API_URL` apuntando a Render.
6. Se restringió `CORS_ORIGIN` (queda pendiente confirmarlo) y se rotaron las contraseñas de `admin` y `secretario`.
7. Se verificó con `/health`, `test:panel`, `test:reportes` y `test:rabbitmq`.
8. Se documentaron las URLs y el procedimiento en el README y en la bitácora.

Los tropiezos de ese camino (permisos del repositorio, worker sin plan gratuito, Vercel multi-servicio, fechas fijas del seed) están en la sección 8.

13.1 Arquitectura del despliegue

Arquitectura de despliegue — producción en plan gratuito



También en vectorial: `img/diagrama-despliegue.svg`.

Componente	Servicio	Plan	Función
Frontend	**Vercel**	Hobby (free)	Sirve el build estático de React/Vite y usa `VITE_API_URL`.
API + worker	**Render**	Web Service free	Ejecuta Express, Socket.IO y el worker de RabbitMQ en un solo proceso.
Base de datos	**Neon**	Free	PostgreSQL administrado con `pgvector`, conexión *pooled* y SSL.
Mensajería	**CloudAMQP**	Little Lemur (free)	Broker AMQP con exchange, colas, reintentos y fallidas.
Correo	**Ethereal**	Free	SMTP de pruebas; los mensajes se leen en su web.
Código	**GitHub**	Free	Repositorio original y fork de despliegue.
IA (opcional)	**Anthropic / Voyage AI**	Pago por uso	Agente Claude y embeddings; el sistema funciona sin ellas en modo local.

13.2 URLs públicas

Recurso	URL
Frontend (panel)	https://sistema-gestion-de-vuelos.vercel.app
API	https://sistema-vuelos-api.onrender.com
Salud de la API	https://sistema-vuelos-api.onrender.com/health
Socket.IO	`https://sistema-vuelos-api.onrender.com` (mismo host, `path: /socket.io`)
Repositorio original (entregable)	https://github.com/Demybrictany/Sistema-gestion-de-vuelos
Fork de despliegue	https://github.com/jjemd101191/Sistema-gestion-de-vuelos
Panel de Render	https://dashboard.render.com
Panel de Vercel	https://vercel.com/dashboard
Consola de Neon	https://console.neon.tech
Panel de CloudAMQP	https://customer.cloudamqp.com
Bandeja de Ethereal	https://ethereal.email/messages
Consola de Anthropic (opcional)	https://console.anthropic.com
Consola de Voyage AI (opcional)	https://dash.voyageai.com

13.3 Neon — base de datos

Proyecto PostgreSQL administrado en AWS `us-east-2`, con la extensión `vector` habilitada. La API usa la cadena **pooled**; el esquema y las migraciones se aplican con la cadena **directa**.

Dato	Valor
Proveedor	Neon (plan Free)
Región	`us-east-2` (AWS)
Base de datos	`neondb`
Usuario	`neondb_owner`
Contraseña registrada	`npg_xrnDSi7M5uJT`

Dato	Valor
Host pooled	`ep-noisy-water-b4ymj35w-pooler.c-6.us-east-2.aws.neon.tech`
Host directo	`ep-noisy-water-b4ymj35w.c-6.us-east-2.aws.neon.tech`
SSL	`sslmode=require` (la cadena pooled añade `channel_binding=require`)
Extensiones	`btree_gist`, `vector`
Contenido	20 tablas, 11 vistas BI y el corpus RAG (4 documentos, 13 fragmentos)

Cadena pooled (la usa Render):

```
postgresql://neondb_owner:npg_xrnDSi7M5uJT@ep-noisy-water-b4ymj35w-pooler.c-6.us-east-2.aws.neon.tech/neondb?sslmode=require&channel_binding=require
```

Cadena directa (solo para crear esquema, migrar o ingestar):

```
postgresql://neondb_owner:npg_xrnDSi7M5uJT@ep-noisy-water-b4ymj35w.c-6.us-east-2.aws.neon.tech/neondb?sslmode=require
```

> El host directo es el mismo de la cadena pooled sin el sufijo `-pooler`. Si Neon cambió el *endpoint*, copia la cadena vigente en **Neon → Connect → Direct connection**.

Carga inicial o reinicio de la base de demostración (PowerShell, desde la raíz):

```
$env:DATABASE_URL="postgresql://neondb_owner:npg_xrnDSi7M5uJT@ep-noisy-water-b4ymj35w.c-6.us-east-2.aws.neon.tech/neondb?sslmode=require"
$env:DB_SSL="true"

npm run db:init          # esquema + seed relativo + vistas (DESTRUCTIVO)
npm run db:migrate       # solo si la base ya existía antes de v1.5.0
npm run db:migrate:agente
npm run rag:ingestar     # 4 documentos → 13 fragmentos

Remove-Item Env:DATABASE_URL
Remove-Item Env:DB_SSL
```

> `db:init` **borra y recrea** todas las tablas. No lo ejecutes contra una base con datos que debas conservar. Neon suspende la base tras unos minutos de inactividad; la primera consulta la despierta.

13.4 Render — API y worker

Dato	Valor
Servicio	`sistema-vuelos-api`
Tipo	Web Service (`runtime: node`)
Plan	Free
Región	Virginia (`virginia`)
Build	`npm ci`
Start	`npm run start:prod`
Health check	`/health`
Auto deploy	Sí (`autoDeploy: true`)
Rama desplegada	`main` (verificar en *Settings → Build & Deploy*)
Repositorio conectado	Fork `jjemd101191/Sistema-gestion-de-vuelos`
Node	22

`render.yaml` es el *blueprint* y declara todas las variables. Las marcadas con `sync: false` se capturan en el panel y no viven en el repositorio.

Variable	Valor / origen
`NODE_VERSION`	`22`
`NODE_ENV`	`production`
`TZ`	`America/Mexico_City`
`DATABASE_URL`	Cadena **pooled** de Neon (secreto, `sync: false`).
`DB_SSL`	`true`
`JWT_SECRET`	Generado por Render (`generateValue: true`); visible en *Environment*.
`JWT_EXPIRES_IN`	`8h`
`CORS_ORIGIN`	Dominio de Vercel; el valor documentado quedó en `*` y está pendiente restringirlo.
`RABBITMQ_URL`	AMQP URL de CloudAMQP (secreto, `sync: false`).
`RABBITMQ_EXCHANGE`	`sistema.vuelos`
`RABBITMQ_EXCHANGE_FALLIDAS`	`sistema.vuelos.fallidas`
`RABBITMQ_QUEUE`	`notificaciones.pasajeros`
`RABBITMQ_QUEUE_FALLIDAS`	`notificaciones.fallidas`
`RABBITMQ_QUEUE_REINTENTOS`	`notificaciones.reintentos`
`RABBITMQ_MAX_REINTENTOS`	`3`
`RABBITMQ_TTL_REINTENTO_MS`	`5000`
`RABBITMQ_PREFETCH`	`10`
`WORKER_REINTENTO_MS`	`5000`
`AGENTE_PROVEEDOR`	`local`

Variable	Valor / origen
`AGENTE_MODELO`	`claude-opus-5-5`
`ANTHROPIC_API_KEY`	Vacía (agente local). Se captura solo si se activa Claude.
`VOYAGE_API_KEY`	Vacía (RAG por texto completo).
`CORREO_MODO`	`simulado` en el blueprint; las credenciales SMTP están cargadas y el cambio a `smtp` está pendiente.
`SMTP_HOST`	Ethereal (`smtp.ethereal.email`), secreto.
`SMTP_PORT`	`587`
`SMTP_SECURE`	`false`
`SMTP_USER`	Cuenta de Ethereal (secreto).
`SMTP_PASS`	Contraseña de Ethereal (secreto).
`SMTP_FROM`	Remitente configurado en Render.

Respuesta esperada de `/health`:

```
{"estado":"ok","base_datos":"ok","mensajeria":"ok","cola":"notificaciones.pasajeros"}
```

Redesplegar: panel de Render → servicio → **Manual Deploy** → **Deploy latest commit** (o *Clear build cache & deploy*). Con `autoDeploy` activo, un push a la rama conectada también dispara el despliegue. Los logs de la API y del worker están en **Logs**; las líneas del worker empiezan con `[worker]`.

13.5 Vercel — frontend

Dato	Valor
Proyecto	`sistema-gestion-de-vuelos`
Plan	Hobby (free)
Root Directory	`frontend`
Framework	Vite
Build Command	`npm run build`
Output Directory	`dist`
Repositorio conectado	Fork `jjemd101191/Sistema-gestion-de-vuelos`
Variable	`VITE_API_URL=https://sistema-vuelos-api.onrender.com` (sin barra final)

`frontend/vercel.json` define el framework, el build, la salida y la reescritura SPA:

```
{
  "framework": "vite",
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

> `VITE_API_URL` se incrusta en el bundle durante el build: si cambia, hay que **redesplegar** el frontend. En local se deja vacía y Vite hace proxy de `/api` y `/socket.io` hacia `http://localhost:3000`.

13.6 CloudAMQP — RabbitMQ

Dato	Valor
Plan	Little Lemur (free)
URL de conexión	`amqps://usuario:password@host/vhost` (en CloudAMQP → *Details* → *AMQP URL*)
Exchange principal	`sistema.vuelos` (topic, durable)
Exchange de fallidas	`sistema.vuelos.fallidas` (direct, durable)
Cola principal	`notificaciones.pasajeros`
Cola de reintentos	`notificaciones.reintentos` (TTL 5 s, vuelve al exchange)
Cola de fallidas	`notificaciones.fallidas`
Binding	`notificacion.*` (publica `notificacion.creada`)
Reintentos	3 (`RABBITMQ_MAX_REINTENTOS`)
Prefetch	10

Flujo: la API publica en `sistema.vuelos` **después del commit**; el worker consume `notificaciones.pasajeros`, envía el correo o lo simula y registra la entrega. Los fallos temporales pasan por `notificaciones.reintentos`; tras 3 intentos van a `notificaciones.fallidas`.

> La AMQP URL es secreta y **no está en el repositorio**. Se copia del panel de CloudAMQP y se pega en Render (`RABBITMQ_URL`) y en el `.env` local si se usa el broker en la nube. El plan gratuito limita conexiones y mensajes.

13.7 Ethereal — SMTP de pruebas

Dato	Valor
Host	`smtp.ethereal.email`
Puerto	`587`
Secure	`false` (STARTTLS)
Usuario y contraseña	Se generan al crear la cuenta; se copian a Render (`SMTP_USER`, `SMTP_PASS`).

Dato	Valor
Bandeja	https://ethereal.email/messages
Modo	`CORREO_MODO=smtp` para enviar; `simulado` para solo registrar la entrega.

Ethereal **no entrega a buzones reales**: es una bandeja de pruebas que demuestra el envío SMTP de punta a punta. Para entrega real se usa Gmail con contraseña de aplicación (`smtp.gmail.com`, 587, `SMTP_SECURE=false`) o Mailtrap.

13.8 GitHub — repositorio y fork

Dato	Valor
Original (entregable)	`https://github.com/Demybrictany/Sistema-gestion-de-vuelos.git`
Fork (despliegue)	`https://github.com/jjemd101191/Sistema-gestion-de-vuelos.git`
Rama estable	`main`
Ramas de trabajo	`juanjo`, `Demy`, `feature/v1.x.x-tema`
Tags presentes	`v1.1.0`, `v1.1.1`, `v1.2.0`, `v1.3.0`, `v1.4.0`, `v1.5.1`, `v1.6.0`, `v1.6.1`
PR abiertos	#12 (v1.6.0) y #13 (v1.6.1, apilado sobre #12)

El repositorio original es el entregable; Render y Vercel leen el **fork** porque la cuenta de despliegue no era dueña del repositorio. Para publicar un cambio:

```
git push origin <rama>     # entregable
git push fork <rama>       # fork que despliega
git push fork --tags       # tags de release
```

Tras fusionar los PR, el servicio de Render y el proyecto de Vercel deben apuntar a `main` para que el despliegue automático tome la versión estable.

13.9 Variables de entorno

Producción (Render). Ver la tabla de la sección 13.4. Las variables `sync: false` (`DATABASE_URL`, `RABBITMQ_URL`, `CORS_ORIGIN`, `ANTHROPIC_API_KEY`, `VOYAGE_API_KEY`, `SMTP_HOST`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM`) se capturan en el panel; `.env` nunca se sube al repositorio.

Desarrollo local (`.env` y Docker Compose):

Variable	Valor local
`DB_NAME`	`aeropuerto_db`
`DB_USER`	`postgres`
`DB_PASSWORD`	`postgres`
`DB_HOST`	`localhost`
`DB_PORT`	`5433` (para no chocar con el PostgreSQL nativo del 5432)
`DB_SSL`	`false`

Variable	Valor local
`PORT`	`3000`
`JWT_SECRET`	`dev-secreto-local-cambiar-en-produccion` (solo local)
`JWT_EXPIRES_IN`	`8h`
`RABBITMQ_URL`	`amqp://admin:admin123@localhost:5672`
`RABBITMQ_EXCHANGE`	`sistema.vuelos`
`CORREO_MODO`	`simulado`

Servicios de `docker-compose.yml`:

Servicio	Imagen	Puertos	Credenciales
PostgreSQL	`pgvector/pgvector:pg16`	`5433:5432`	`postgres` / `postgres` / base `aeropuerto_db`
RabbitMQ	`rabbitmq:3.13-management-alpine`	`5672`, `15672`	`admin` / `admin123` (panel en http://localhost:15672)

13.10 APIs y endpoints

Base: `https://sistema-vuelos-api.onrender.com/api` (local: `http://localhost:3000/api`). Todas las rutas salvo el login requieren `Authorization: Bearer <JWT>`.

Grupo	Rutas principales	Roles
Autenticación	`POST /api/auth/login`, `GET /api/auth/me`	Público / autenticado
Roles	`/api/roles`	ADMIN
Usuarios	`/api/usuarios`	ADMIN
Pasajeros	`/api/pasajeros`	ADMIN, SECRETARIO
Aeropuertos	`/api/aeropuertos`	ADMIN, SECRETARIO
Puertas	`/api/puertas`	ADMIN, SECRETARIO
Bandas	`/api/bandas-equipaje`	ADMIN, SECRETARIO
Vuelos	`GET/POST /api/vuelos`, `GET/PUT/DELETE /api/vuelos/:id`	Por método
Vuelos (acciones)	`PATCH /api/vuelos/:id/estado`, `/reprogramar`, `/puerta`; `POST /api/vuelos/:id/cancelar`	ADMIN, SECRETARIO
Vuelos del pasajero	`GET /api/vuelos/mios`	PASAJERO
Itinerarios	`/api/itinerarios`	ADMIN, SECRETARIO
Reservas	`/api/pasajero-vuelo`	ADMIN, SECRETARIO
Escalas	`/api/escalas`	ADMIN, SECRETARIO
Equipajes	`/api/equipajes`, `POST /:id/tramos`, `PATCH /:id/banda`, `PATCH /:id/estado`	Por método
Equipaje-vuelo	`/api/equipaje-vuelo`	ADMIN, SECRETARIO
Notificaciones	`/api/notificaciones`, `/mias`, `/entregas`, `/entregas/resumen`, `/:id/entregas`	Por método

Grupo	Rutas principales	Roles
Historiales	`/api/historial-vuelo`, `/api/historial-equipaje`	ADMIN, SECRETARIO
Auditoría	`/api/auditoria`	ADMIN
Reportes	`/api/reportes`, `/api/reportes/:clave`, `/api/reportes/:clave/exportar?formato=excel` o `?formato=pdf`	ADMIN, SECRETARIO
Panel	`GET /api/panel`	ADMIN, SECRETARIO
Agente	`POST /api/agente/consultas`; `GET /api/agente/interacciones`, `/documentos`; `POST /api/agente/ingestar`	Consulta: autenticados; resto: ADMIN
Salud	`GET /health` (fuera de `/api`)	Público

Ejemplos:

```
# Login
curl -X POST https://sistema-vuelos-api.onrender.com/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"nombre_usuario":"admin","password":"<clave>"}'

# Panel (reemplaza $TOKEN por el JWT devuelto)
curl https://sistema-vuelos-api.onrender.com/api/panel \
  -H "Authorization: Bearer $TOKEN"

# Reporte en Excel
curl -o ocupacion.xlsx \
  "https://sistema-vuelos-api.onrender.com/api/reportes/ocupacion-por-vuelo/exportar?formato=excel" \
  -H "Authorization: Bearer $TOKEN"

# Agente
curl -X POST https://sistema-vuelos-api.onrender.com/api/agente/consultas \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"pregunta":"¿Qué vuelos salen hoy y por qué puerta?"}'
```

Socket.IO: el cliente se conecta al mismo host de la API y escucha `panel:actualizar`; al recibirlo vuelve a pedir `GET /api/panel`.

```
import { io } from 'socket.io-client';

const socket = io('https://sistema-vuelos-api.onrender.com', {
  auth: { token: jwt },
});
socket.on('panel:actualizar', () => recargarPanel());
```

APIs externas (todas opcionales):

Servicio	Uso	Clave	Estado
Anthropic Messages API	Agente con tool use (`claude-opus-5-5`).	`ANTHROPIC_API_KEY`	No configurada; el agente usa el modo local.
Voyage AI	Embeddings de 1024 dimensiones para pgvector.	`VOYAGE_API_KEY`	No configurada; el RAG usa texto completo en español.
Ethereal SMTP	Entrega de correos de prueba.	`SMTP_USER` , `SMTP_PASS`	Cargadas en Render.
CloudAMQP	Broker AMQP.	`RABBITMQ_URL`	Configurada en Render.
Neon	PostgreSQL.	`DATABASE_URL`	Configurada en Render.

Errores: formato `{ "mensaje": "...", "errores": [] }` ; códigos 200, 201, 400, 401, 403, 404, 409, 429 y 500.

13.11 Verificación en producción

```
# 1. Salud
curl.exe https://sistema-vuelos-api.onrender.com/health

# 2. Mensajería de punta a punta (crea y borra una notificación de prueba)
$env:API_URL="https://sistema-vuelos-api.onrender.com"
$env:ADMIN_USUARIO="admin"
$env:ADMIN_PASSWORD="<clave vigente>"
npm run test:rabbitmq
Remove-Item Env:API_URL, Env:ADMIN_USUARIO, Env:ADMIN_PASSWORD

# 3. Panel y reportes contra producción (solo lectura)
npm run test:panel
npm run test:reportes
```

Resultados de referencia en la última verificación: panel **14/14**, reportes **29/29**, RabbitMQ **10/10** y `/health` en `ok` . **No ejecutes `test:api` contra producción**: crea y modifica datos.

13.12 Operación del plan gratuito

- **Render** duerme el servicio tras ~15 minutos sin tráfico. La primera visita puede tardar 30–60 s y, mientras duerme, el worker no consume la cola.
- Los mensajes que llegan mientras duerme **quedan en CloudAMQP** y se procesan al despertar; no se pierden.
- **Neon** suspende la base por inactividad; la primera consulta la despierta.
- **Vercel** sirve el frontend estático sin suspensión.
- Para la defensa, abre el panel unos minutos antes. Opcional: un monitor gratuito (UptimeRobot) que consulte `/health` cada 5–10 minutos mantiene Render despierto.
- El worker comparte proceso con la API porque el plan free no incluye *Background Workers*. `scripts/iniciar-produccion.js` reinicia el worker si se cae y detiene todo si la API termina.

13.13 Seguridad y credenciales

> **Advertencia:** esta sección contiene la cadena de conexión real de la base de demostración. No publiques este manual fuera del equipo. Si se comparte, rota la contraseña de Neon, actualiza `DATABASE_URL` en Render y en el `.env` local, y redespliega.

Dato	Valor / ubicación
Cadena pooled de Neon	Ver sección 13.3. En Render: `DATABASE_URL`.
Cadena directa de Neon	Solo para `db:init`, migraciones e ingesta.
Contraseña de Neon	`npg_xrnDSi7M5uJT` (rotar si se filtra).
`JWT_SECRET` de producción	Generado por Render; se consulta en *Environment*, no está en el repo.
AMQP URL de CloudAMQP	Se consulta en *CloudAMQP → Details → AMQP URL*; está en Render como `RABBITMQ_URL`.
Usuario y contraseña SMTP	Se consultan en Ethereal y en Render (`SMTP_USER`, `SMTP_PASS`).
`ANTHROPIC_API_KEY` / `VOYAGE_API_KEY`	Vacías. Si se activan, se capturan solo en Render.
Contraseñas de `admin` y `secretario` en producción	Rotadas con `npm run db:password`; no quedaron registradas en el repositorio. La guía usó como ejemplo `UnaClaveAdmin2026!` y `UnaClaveSecretario2026!`; confirma con el equipo cuál está vigente.
Usuarios del seed local	`admin` / `Admin123!`, `secretario` / `Secretario123!`, `carlos`, `sofia`, `andrea` / `Pasajero123!`. Solo aplican a la base local.

Checklist mínimo antes de publicar:

- [x] `JWT_SECRET` generado por Render, distinto del de desarrollo.
- [x] Contraseñas del seed rotadas en producción.
- [ ] `CORS_ORIGIN` restringido al dominio de Vercel (pendiente).
- [x] `.env` fuera del repositorio (`.gitignore`).
- [x] Credenciales solo en los paneles de Render, Vercel, Neon y CloudAMQP.
- [x] HTTPS automático en Render y Vercel.
- [x] `npm run db:password -- <usuario> "<clave>"` documentado para rotar.

13.14 Problemas comunes

Síntoma	Causa probable	Solución
`/health` responde 503	`DATABASE_URL` incorrecta o `DB_SSL` distinto de `true`.	Usa la cadena pooled de Neon y `DB_SSL=true`.
`mensajeria: "configurada"` y no `"ok"`	CloudAMQP rechazó la conexión.	Verifica la AMQP URL, el *vhost* y el límite de conexiones.
El frontend muestra error de red o CORS	`VITE_API_URL` vacía o `CORS_ORIGIN` sin el dominio de Vercel.	Corrige la variable y redespliega el frontend.
El login falla con 401	Contraseña rotada.	Usa la vigente o define `ADMIN_PASSWORD` en las pruebas.

Síntoma	Causa probable	Solución
El worker no registra entregas	Servicio dormido, `RABBITMQ_URL` vacía o SMTP sin credenciales.	Revisa los logs `[worker]` y la configuración de correo.
El correo queda `SIMULADA`	`CORREO_MODO=simulado`.	Cambia a `smtp` y redespliega.
Build de Render falla en `npm ci`	`package-lock.json` desincronizado.	Ejecuta `npm install` en local y sube el lockfile.
El panel tarda la primera vez	Render estaba dormido.	Espera el arranque o usa un monitor de disponibilidad.
Los datos de la demo se ven viejos	La base no se reinició o el seed no es reciente.	Ejecuta `db:init` (destructivo) con la cadena directa.

---

14. Pendientes

- Aprobar y fusionar los PR #12 y #13.
- Redesplegar Render con la release v1.6.x.
- Restringir `CORS_ORIGIN` al dominio de Vercel.
- Crear el tablero `.pbix` de Power BI.
- Crear los tags de release en `main`.
- Opcional: activar Claude con `ANTHROPIC_API_KEY`.
- Opcional: vista de pasajero y acciones operativas en el frontend.
- Opcional: pruebas unitarias y CI.

Documentos relacionados: `01-MANUAL_DE_USUARIO.md`, `02-MANUAL_ADMINISTRADOR_PROGRAMADOR.md`, `docs/BITACORA.md`, `docs/MANUAL_TECNICO.md`, `CLAUDE.md` y `AGENTS.md`.